Nama: M. Rivaldi Juliadin

Nim: 13020240021

Kelas: A1.TI

Dosen Pengampu: Mardiyyah Hasnawi, S.Kom., M.T., MTA.

1. Penjelasan Kode Program Nomor 1
   Tujuan utama program ini adalah untuk menunjukkan bagaimana cara menyimpan angka desimal ke dalam dua jenis variabel yang berbeda, yaitu float dan double.Program ini juga memperlihatkan implicit casting (konversi otomatis), di mana nilai bertipe float dimasukkan ke dalam variabel bertipe double.

```
public class Asgdll {
   public static void main(String[] args) {
```

Penjelasan:

- public class Asgdll { : Wadah utama program. Nama kelas harus sama dengan nama file. Keyword public berarti kelas ini bisa diakses dari mana saja.
- public static void main(String[] args) { : Titik awal (entry point) saat program dijalankan. Tanpa ini, kode tidak bisa dieksekusi.

Tujuan : Sebagai wadah dari program dan menjadi titik awal saat program di jalankan

Deklarasi Variabel

- float f= 20.0f : Digunakan untuk bilangan desimal presisi tunggal (32-bit). Akhiran f wajib ada untuk menandakan bahwa angka tersebut adalah float.
- double fll: Digunakan untuk bilangan desimal presisi ganda (64-bit). Ini adalah standar default Java untuk angka desimal.
- fll=10.0f : Mengisi variabel fll dengan angka 10.0. Meskipun 10.0 bertipe float, Java mengizinkannya masuk ke double karena kapasitas double lebih besar.

System.out.println ("f : "+f+ "\nf11: "+fll) : adalah perintah standar Java untuk mencetak teks ke layar sekaligus pindah baris di akhir. Bertujuan untuk menampilkan output pada layar

Output:

```
f : 20.0
f11: 10.0
```

2. Penjelasan kode program nomor 2
   Tujuan program ini adalah untuk meperkenalkan konsep dasar penggunaan variable di java.Serta, menjelaskan penggunaan print dan println.

```
public class Asign {
    public static void main(String[] args) {
```

Penjelasan:

- public class Asign { : Wadah utama program. Nama kelas harus sama dengan nama file. Keyword public berarti kelas ini bisa diakses dari mana saja.
- public static void main(String[] args) { : Titik awal (entry point) saat program dijalankan. Tanpa ini, kode tidak bisa dieksekusi

Tujuan : Sebagai wadah dari program dan menjadi titik awal saat program di jalankan

Deklarasi Variabel

```
Int i;
```

Penjelasan:

- Int adalah Tipe data untuk menyimpan integer (bilangan bulat) tanpa desimal.
- i adalah Nama variabel (identifier) yang digunakan untuk menyimpan data.

```
System.out.print ("hello\n"); i = 5;
System.out.println ("Ini nilai i : " + i);
```

Penjelasan:

- System.out.print ("hello\n"); i = 5 : Terdapat 2 statement dalam 1 baris yang dipisahkan oleh titik koma(;) . System.out.print("hello\n") mencetak teks "hello" lalu pindah baris karena \n . i=5 (variable I diberi nilai 5)
- System.out.println ("Ini nilai i : " + i); : Mencetak teks "ini nilai i: " yang digabung dengan nilai variable i. + I (penggabungan string dengan nilai integer secara otomatis)

Perbandingan

print vs println:

- System.out.print() = cetak teks, kursor tetap di baris yang sama
- System.out.println() = cetak teks, kursor otomatis pindah baris

Output

```
hello
Ini nilai i : 5
```

3. Penjelasan program nomor 3

   Program ini bertujuan untuk mendemonstrasikan berbagai tipe data primitif dalam Java secara menyeluruh, mencakup tipe data bilangan bulat (short, int, long), karakter (char), dan bilangan desimal (double, float). Program juga memperlihatkan dua cara inisialisasi char yaitu menggunakan nilai integer dan karakter langsung, serta bagaimana Java menampilkan setiap tipe data tersebut ke console.

```
public class ASIGNi {
   public static void main(String[] args) {
```

Penjelasan:

- public class ASIGNi { : Wadah utama program. Nama kelas harus sama dengan nama file. Keyword public berarti kelas ini bisa diakses dari mana saja.
- public static void main(String[] args) { : Titik awal (entry point) saat program dijalankan. Tanpa ini, kode tidak bisa dieksekusi

```
short ks = 1;
int ki = 1;
long kl = 10000;
```

Penjelasan:

Ketiga variabel ini semuanya menyimpan bilangan bulat, namun dengan kapasitas yang berbeda. short berukuran 16-bit dan mampu menyimpan nilai dari -32.768 hingga 32.767, cocok digunakan ketika memori sangat terbatas. int berukuran 32-bit dengan rentang sekitar -2,1 miliar hingga 2,1 miliar dan merupakan tipe bilangan bulat default yang paling umum digunakan di Java. long berukuran 64-bit dan digunakan ketika nilai yang disimpan melebihi kapasitas int, seperti nilai populasi dunia atau timestamp dalam milidetik. Normalnya long memerlukan suffix L di belakang nilainya (contoh: 10000L), namun karena nilai 10000 masih dalam rentang int, Java mengizinkan tanpa suffix.

- char c = 65;
- char c1 = 'Z';

Penjelasan:

Variabel c dan c1 keduanya bertipe char, yaitu tipe data untuk menyimpan satu karakter Unicode berukuran 16-bit. Yang menarik di sini adalah dua cara berbeda dalam menginisialisasi char. Variabel c diisi dengan angka integer 65, yang merupakan nilai ASCII/Unicode dari huruf 'A'. Ketika dicetak, Java akan menampilkan karakternya yaitu

A, bukan angka 65. Sementara c1 diinisialisasi langsung dengan karakter 'Z' menggunakan tanda kutip tunggal.

- double x = 50.2f;
- float y = 50.2f;

Penjelasan:

Kedua variabel ini menyimpan bilangan desimal. Variabel y bertipe float (32-bit) dan menerima nilai 50.2f secara langsung sesuai tipenya. Variabel x bertipe double (64-bit) namun diisi dengan literal float (50.2f). Java secara otomatis melakukan widening conversion dari float ke double, sehingga nilai 50.2f diperluas menjadi representasi double. Akibatnya nilai x saat dicetak akan sedikit berbeda dari y karena perbedaan presisi antara kedua tipe tersebut.

- System.out.println("Karakter = "+ c);
- System.out.println("Karakter = "+ c1);

Penjelasan:
Kedua baris ini mencetak variabel char. Meskipun c diinisialisasi dengan angka 65, ketika digabungkan dengan String menggunakan operator +, Java secara otomatis mengonversinya menjadi representasi karakternya yaitu A. Begitu pula c1 yang akan mencetak Z.

- System.out.println("Bilangan integer (short) = "+ ks);
- System.out.println("\t(int) = "+ ki);
- System.out.println("\t(long)= "+ kl);

Penjelasan:

Tiga baris ini mencetak nilai bilangan bulat. Yang menarik adalah penggunaan escape sequence \t pada baris kedua dan ketiga, yang berfungsi menyisipkan karakter tab horizontal sehingga output terlihat menjorok ke dalam dan lebih rapi secara visual di console.

- System.out.println("Bilangan Real x = "+ x);
- System.out.println("Bilangan Real y = "+ y);

Penjelasan:
Kedua baris ini mencetak nilai desimal. Karena x adalah double hasil konversi dari float, nilainya mungkin tampil sebagai 50.20000076293945 (mengikuti representasi floating point IEEE 754), sedangkan y yang murni float akan tampil sebagai 50.2.

Output:

```
Karakter = A
Karakter = Z
Karakter = A
Karakter = Z
Bilangan integer (short) = 1
        (int) = 1
        (long)= 10000
Bilangan Real x = 50.20000076293945
Bilangan Real y = 50.2
```

4. Penjelasan program nomor 4

   Program ini bertujuan untuk mendemonstrasikan cara membaca input dari pengguna melalui keyboard menggunakan class Scanner dari library java.util. Program meminta pengguna mengetik sebuah bilangan bulat, membacanya, lalu menampilkannya kembali ke console. Ini adalah konsep dasar interaksi antara program dan pengguna (input/output) dalam Java.

   - import java.util.Scanner;

   Penjelasan:

   Baris ini adalah pernyataan import yang memberitahu Java untuk menggunakan class Scanner dari package java.util. Package java.util adalah bagian dari Java Standard Library yang berisi berbagai utilitas bawaan. Tanpa baris import ini, Java tidak akan mengenali kata Scanner dan program akan gagal dikompilasi. Keyword import harus selalu ditulis di luar dan sebelum deklarasi kelas

   - public class BacaData {
     public static void main(String[] args) {

   Penjelasan:

   - public class BacaData{ : Wadah utama program. Nama kelas harus sama dengan nama file. Keyword public berarti kelas ini bisa diakses dari mana saja.
   - public static void main(String[] args) { : Titik awal (entry point) saat program dijalankan. Tanpa ini, kode tidak bisa dieksekusi

- int a;
  Scanner masukan;

Penjelasan:

Variabel a bertipe int (bilangan bulat 32-bit) yang nantinya akan menampung nilai yang diketik oleh pengguna. Variabel masukan bertipe Scanner, yang berbeda dari variabel-variabel sebelumnya karena Scanner bukan tipe data primitif melainkan sebuah class (tipe data referensi/objek). Pada tahap ini masukan baru dideklarasikan, belum ada objek Scanner yang dibuat.

- System.out.print("Contoh membaca dan menulis, ketik nilai integer: \n");

  Penjelasan:
  Baris ini menggunakan System.out.print (tanpa ln) untuk mencetak pesan instruksi ke layar.

- masukan = new Scanner(System.in);

  Penjelasan:

  Ini adalah baris terpenting dalam program. Keyword new digunakan untuk membuat objek baru dari class Scanner. System.in adalah objek bawaan Java yang merepresentasikan input standar dari keyboard. Jadi new Scanner(System.in) artinya membuat sebuah Scanner yang siap membaca data dari keyboard, lalu menyimpan referensinya ke variabel masukan. Proses ini disebut instansiasi objek.

- a = masukan.nextInt();

  Penjelasan:

  Method nextInt() milik objek masukan digunakan untuk membaca satu bilangan bulat yang diketik pengguna dan menekan Enter. Nilai yang dibaca langsung disimpan ke variabel a. Program akan berhenti sementara (blocking) di baris ini menunggu pengguna selesai mengetik. Jika pengguna mengetik bukan angka (misalnya huruf), program akan melempar InputMismatchException dan crash.

- System.out.print("Nilai yang dibaca : "+ a);

  Penjelasan:

  Setelah input berhasil dibaca dan disimpan di a, baris ini mencetak hasilnya ke console dengan menggabungkan String dan nilai variabel a menggunakan operator +.

- masukan.close();

Penjelasan:

Method close() digunakan untuk menutup objek Scanner dan melepaskan resource sistem yang digunakan. Ini adalah praktik pemrograman yang baik (best practice) karena Scanner yang terhubung ke System.in menggunakan resource I/O sistem. Jika tidak ditutup, dapat menyebabkan memory leak pada program yang lebih besar dan kompleks. Output:

```
Contoh membaca dan menulis, ketik nilai integer:
021
Nilai yang dibaca : 21
```

5. Penjelasan program nomor 5

   Program ini bertujuan untuk mendemonstrasikan cara membaca input dari keyboard menggunakan class BufferedReader dan InputStreamReader sebagai alternatif dari Scanner. Program membaca satu karakter dan satu bilangan bulat dari pengguna, lalu menampilkannya kembali. Selain itu, program juga memperlihatkan dua cara penulisan inisialisasi BufferedReader dan menyisipkan komentar kode dari pendekatan GUI menggunakan JOptionPane.

   - import java.io.BufferedReader;
   - import java.io.IOException;
   - import java.io.InputStreamReader;
   - //import javax.swing.*;

   Penjelasan:

   Terdapat tiga import aktif dan satu import yang dinonaktifkan. Package java.io adalah library Java yang menangani operasi Input/Output (I/O), berbeda dari java.util yang bersifat utilitas umum. BufferedReader diimport untuk membaca teks dari input stream secara efisien menggunakan buffer. IOException diimport karena operasi I/O berpotensi gagal (misalnya stream terputus) sehingga exception ini wajib ditangani. InputStreamReader diimport sebagai jembatan yang mengubah byte stream menjadi character stream yang bisa dibaca BufferedReader.

   Baris //import javax.swing.*; dinonaktifkan dengan komentar //. Package javax.swing berisi komponen GUI Java seperti JOptionPane untuk dialog pop-up. Tanda * artinya wildcard import yang mengimport semua class dalam package tersebut sekaligus. Karena dikomentari, fitur GUI tidak aktif dalam program ini.

- public class Bacakar {
  public static void main(String[] args) throws IOException {

Penjelasan:

- public class Bacakar{ : Wadah utama program. Nama kelas harus sama dengan nama file. Keyword public berarti kelas ini bisa diakses dari mana saja.
- public static void main(String[] args) throws IOException{ : Titik awal (entry point) saat program dijalankan. Tanpa ini, kode tidak bisa dieksekusi, Berbeda dari program sebelumnya, method main di sini menambahkan throws IOException. Ini adalah deklarasi bahwa method ini berpotensi melempar IOException dan tidak menanganinya sendiri, melainkan meneruskannya ke pemanggil (dalam hal ini JVM). Ini diperlukan karena method readLine() dari BufferedReader secara eksplisit dapat melempar IOException, dan Java mewajibkan semua checked exception untuk ditangani atau dideklarasikan.

- char cc; int bil;

Penjelasan:

Dua variabel dideklarasikan dalam satu baris, dipisahkan titik koma. cc bertipe char untuk menyimpan satu karakter hasil input, dan bil bertipe int untuk menyimpan bilangan bulat. Penulisan dua deklarasi dalam satu baris seperti ini valid di Java namun kurang disarankan karena mengurangi keterbacaan kode.

- InputStreamReader isr = new InputStreamReader(System.in);
  BufferedReader dataIn = new BufferedReader(isr);

Penjelasan:

Cara pertama menggunakan dua langkah terpisah. Pertama, objek InputStreamReader dibuat dengan nama isr, yang bertugas mengubah byte stream dari System.in (keyboard) menjadi character stream yang dapat diproses. Kemudian objek BufferedReader bernama dataIn dibuat dengan menerima isr sebagai sumbernya. BufferedReader menambahkan lapisan buffering sehingga pembacaan lebih efisien karena data dibaca dalam blok bukan per karakter. Cara ini lebih mudah dibaca dan di-debug karena setiap langkah terpisah jelas.

- BufferedReader datAIn = new BufferedReader(new InputStreamReader(System.in));

Penjelasan:

Cara kedua adalah penulisan ringkas (one-liner) yang menggabungkan pembuatan InputStreamReader langsung di dalam konstruktor BufferedReader tanpa menyimpannya ke variabel terpisah. Hasil akhirnya identik dengan cara pertama, hanya penulisannya lebih

singkat. Perhatikan bahwa program ini membuat dua objek BufferedReader (dataIn dan datAIn) yang keduanya terhubung ke System.in. Dalam praktik nyata cukup satu saja.

- System.out.print("hello\n");
  System.out.print("baca 1 karakter : ");
  cc = dataIn.readLine().charAt(0);

Penjelasan:
Setelah mencetak pesan sambutan dan instruksi, program membaca input karakter dengan chaining method dalam satu baris. Method readLine() milik BufferedReader membaca seluruh satu baris teks yang diketik pengguna hingga Enter ditekan, dan mengembalikannya sebagai objek String. Kemudian method charAt(0) dipanggil langsung pada String hasil readLine(), yang mengambil karakter pada indeks ke-0 yaitu karakter pertama dari String tersebut. Hasilnya disimpan ke variabel cc bertipe char. Jika pengguna langsung menekan Enter tanpa mengetik apapun, readLine() mengembalikan String kosong dan charAt(0) akan melempar StringIndexOutOfBoundsException.

- System.out.print("baca 1 bilangan : ");
  bil = Integer.parseInt(datAIn.readLine());

Penjelasan:

Untuk membaca bilangan bulat, program kembali menggunakan readLine() namun kali ini dari objek datAIn. Karena readLine() selalu mengembalikan String, hasilnya tidak bisa langsung disimpan ke int. Maka digunakan Integer.parseInt() yaitu method static dari class wrapper Integer yang mengonversi String menjadi nilai int. Jika pengguna mengetik bukan angka (misalnya huruf), method ini akan melempar NumberFormatException.

- System.out.print(cc + "\n" + bil + "\n");
  System.out.print("bye \n");

Penjelasan:
Program mencetak karakter dan bilangan yang telah dibaca, masing-masing diikuti \n untuk baris baru, lalu mencetak pesan penutup bye.

Output:

```
hello
baca 1 karakter : A
baca 1 bilangan : 2
A
2
bye
```

6. Penjelasan program nomor 6

   Program ini bertujuan untuk mendemonstrasikan konsep Type Casting (konversi tipe data) dalam Java, yaitu proses mengubah nilai dari satu tipe data ke tipe data lain secara eksplisit menggunakan operator cast (tipedata). Program memperlihatkan berbagai kombinasi casting antar tipe primitif seperti int, float, double, dan char, serta menunjukkan bagaimana nilai berubah atau terpotong ketika dikonversi antar tipe yang berbeda ukurannya.

   - public class Casting1 {
     public static void main(String[] args) {

   Penjelasan:

   - public class Casting1 { : Wadah utama program. Nama kelas harus sama dengan nama file. Keyword public berarti kelas ini bisa diakses dari mana saja.
   - public static void main(String[] args) { : Titik awal (entry point) saat program dijalankan. Tanpa ini, kode tidak bisa dieksekusi

   - int a=5, b=6;
     float d=2.f, e=3.2f;
     char g='5';
     double k=3.14;

   Penjelasan:

   Program mendeklarasikan dan langsung menginisialisasi semua variabel dalam satu langkah. Variabel a dan b bertipe int (bilangan bulat 32-bit). Variabel d dan e bertipe float (desimal 32-bit) dengan suffix f. Perhatikan 2.f yang merupakan penulisan singkat dari 2.0f, keduanya valid di Java. Variabel g bertipe char diisi dengan karakter '5' (bukan angka 5), yang dalam tabel ASCII/Unicode memiliki nilai integer 53. Variabel k bertipe double (desimal 64-bit) dengan nilai 3.14, tidak perlu suffix karena double adalah tipe desimal default Java.

   - System.out.println((float)a); // int <-- float : Variabel a bertipe int bernilai 5 dicast ke float. Ini adalah widening casting karena float lebih besar dari int. Tidak ada data yang hilang, nilai 5 menjadi 5.0 dalam representasi float. Output: 5.0
   - System.out.println((double)b); // int <-- double : Variabel b bertipe int bernilai 6 dicast ke double. Juga merupakan widening casting karena double (64-bit) jauh lebih besar dari int (32-bit). Nilai 6 menjadi 6.0 dalam representasi double. Output: 6.0
   - System.out.println((int)d); // float <-- int : Variabel d bertipe float bernilai 2.0 dicast ke int. Ini adalah narrowing casting yang memotong bagian desimal secara langsung (truncation), bukan pembulatan. Nilai 2.0 menjadi 2. Jika nilainya 2.9f sekalipun, hasilnya tetap 2, bukan 3. Output: 2

- System.out.println((double)e); // float <-- double : Variabel e bertipe float bernilai 3.2f dicast ke double. Ini widening casting dari float ke double. Namun karena 3.2 tidak dapat direpresentasikan secara tepat dalam format biner floating point, nilai 3.2f yang disimpan sebenarnya sedikit berbeda dari 3.2 yang tepat. Ketika diperluas ke double, ketidaktepatan ini ikut terbawa dan terlihat. Output: 3.200000047683716
- System.out.println((int)g); // char <-- int (ASCII) : Variabel g bertipe char berisi karakter '5'. Perlu dipahami bahwa karakter '5' berbeda dari angka 5. Dalam tabel ASCII/Unicode, karakter '5' memiliki nilai numerik 53. Ketika dicast ke int, Java mengambil nilai Unicode-nya yaitu 53. Output: 53
- System.out.println((float)g); // char <-- float (ASCII) : Karakter '5' (nilai Unicode 53) dicast ke float. Java pertama mengambil nilai integer Unicode-nya (53) lalu mengonversinya ke representasi float. Output: 53.0
- System.out.println((double)g); // char <-- double (ASCII) : Sama seperti casting ke float, karakter '5' (Unicode 53) dicast ke double. Nilai integer 53 direpresentasikan dalam bentuk double. Output: 53.0
- System.out.println((int)k); // double <-- int : Variabel k bertipe double bernilai 3.14 dicast ke int. Ini narrowing casting yang paling umum dijumpai. Bagian desimal .14 dipotong habis (truncation), hanya bagian bulat yang dipertahankan. Output: 3
- System.out.println((float)k); // double <-- float : Variabel k bertipe double bernilai 3.14 dicast ke float. Ini narrowing casting dari double (64-bit) ke float (32-bit). Karena float memiliki presisi lebih rendah, nilai 3.14 dalam double dibulatkan ke representasi float terdekat. Output: 3.14

Output:

```
5.0
6.0
2
3.200000047683716
53
53.0
53.0
3
3.14
```

7. Penjelasan program nomor 7
   Program ini bertujuan untuk mendemonstrasikan konversi tipe data melibatkan String dan Wrapper Class dalam Java. Berbeda dari Casting1 yang hanya menggunakan operator cast (tipedata) antar tipe primitif, program ini memperlihatkan cara mengonversi String ke tipe primitif, tipe primitif ke String, dan antar tipe numerik menggunakan Wrapper Class seperti

Integer, Double, dan Float. Konsep ini sangat penting dalam pemrograman nyata karena data dari input pengguna selalu diterima dalam bentuk String.

- public class Casting2 {
  public static void main(String[] args) {

Penjelasan:

- public class Casting2{ : Wadah utama program. Nama kelas harus sama dengan nama file. Keyword public berarti kelas ini bisa diakses dari mana saja.
- public static void main(String[] args) { : Titik awal (entry point) saat program dijalankan. Tanpa ini, kode tidak bisa dieksekusi

- int a=8, b=9;
  float d=2.f, e=3.2f;
  char g='5';
  double k=3.14;
  String n="67", m="45", l="100";

Penjelasan:

Program mendeklarasikan dua kelompok variabel. Kelompok pertama adalah tipe primitif biasa (int, float, char, double) yang sudah dikenal dari program sebelumnya. Kelompok kedua adalah variabel bertipe String yaitu n, m, dan l. String bukan tipe primitif melainkan sebuah class bawaan Java dari package java.lang yang diimport otomatis. Nilainya ditulis dengan tanda kutip ganda dan berisi representasi teks dari angka-angka. Nilai awal variabel primitif (a=8, b=9, dst) akan ditimpa oleh hasil konversi di baris-baris berikutnya.

- a = Integer.parseInt(n);
  k = Double.parseDouble(m);
  d = Float.parseFloat(l);

Penjelasan:

Ketiga baris ini menggunakan Wrapper Class untuk mengonversi String menjadi tipe primitif. Wrapper Class adalah class pembungkus yang merepresentasikan tipe primitif sebagai objek dan menyediakan berbagai method utilitas.

Integer.parseInt(n) memanggil method static parseInt() dari Wrapper Class Integer. Method ini mengurai String "67" dan mengonversinya menjadi nilai int primitif 67, lalu disimpan ke variabel a. Nilai awal a=8 tertimpa menjadi 67.

Double.parseDouble(m) memanggil method static parseDouble() dari Wrapper Class Double. String "45" dikonversi menjadi nilai double primitif 45.0, lalu disimpan ke k. Nilai awal k=3.14 tertimpa menjadi 45.0.

Float.parseFloat(l) memanggil method static parseFloat() dari Wrapper Class Float. String "100" dikonversi menjadi nilai float primitif 100.0, lalu disimpan ke d. Nilai awal d=2.f tertimpa menjadi 100.0.

- n = String.valueOf(b);
  m = String.valueOf(g);
  l = String.valueOf(e);

Penjelasan:
Ketiga baris ini menggunakan method static String.valueOf() dari class String untuk mengonversi berbagai tipe data menjadi String. Method valueOf() bersifat overloaded, artinya Java menyediakan versi method ini untuk hampir semua tipe data primitif (int, char, float, double, long, boolean) sehingga cukup memanggil satu nama method yang sama untuk tipe yang berbeda-beda.

String.valueOf(b) mengonversi int b=9 menjadi String "9", disimpan ke n. Nilai n yang sebelumnya "67" tertimpa menjadi "9".

String.valueOf(g) mengonversi char g='5' menjadi String "5", disimpan ke m. Berbeda dari casting char ke int yang menghasilkan nilai ASCII (53), String.valueOf() pada char menghasilkan representasi karakternya langsung yaitu "5".

String.valueOf(e) mengonversi float e=3.2f menjadi String "3.2", disimpan ke l. Nilai l yang sebelumnya "100" tertimpa menjadi "3.2".

- k = Double.valueOf(a).intValue();
  double c = Integer.valueOf(b).doubleValue();

Dua baris ini menggunakan teknik konversi yang berbeda dari parseXxx() karena melibatkan pembuatan objek Wrapper terlebih dahulu lalu memanggil method konversinya. Ini adalah cara lama (sebelum Java 5) yang masih valid hingga sekarang.

Double.valueOf(a) membungkus nilai int a=67 menjadi objek Double (bukan primitif double). Proses ini disebut boxing. Kemudian .intValue() dipanggil pada objek Double tersebut untuk mengambil kembali nilainya sebagai int primitif. Hasilnya 67 (sebagai int) lalu disimpan ke variabel double k. Java secara otomatis mengonversi int ke double (widening), sehingga k menjadi 67.0. Kombinasi Double.valueOf(a).intValue() ini sebenarnya memutar balik — membuat Double lalu mengambil intValue-nya — sehingga hasilnya sama seperti hanya mengassign (double)(int)a.

Double.valueOf(b) → tidak digunakan di sini. Yang digunakan adalah Integer.valueOf(b) yang membungkus int b=9 menjadi objek Integer. Kemudian .doubleValue() dipanggil untuk mengambil nilainya sebagai double primitif. Hasilnya 9.0 disimpan ke variabel double c yang baru dideklarasikan sekaligus.

Output:

```
a : 67
k : 45.0
d : 100.0
n : 9
m : 5
l : 3.2
k : 67.0
c : 9.0
l : 3.2
```

8. Penjelasan program nomor 8

   Program ini bertujuan untuk mendemonstrasikan penggunaan Operator Ternary dalam Java, yaitu operator kondisional ringkas yang dapat menggantikan struktur if-else sederhana dalam satu baris ekspresi. Program membandingkan dua nilai integer dan menampilkan nilai yang lebih kecil di antara keduanya menggunakan ekspresi ternary

   - public class Ekspresi {
     public static void main(String[] args) {

   Penjelasan:

   - public class Ekspresi{ : Wadah utama program. Nama kelas harus sama dengan nama file. Keyword public berarti kelas ini bisa diakses dari mana saja.
   - public static void main(String[] args) { : Titik awal (entry point) saat program dijalankan. Tanpa ini, kode tidak bisa dieksekusi

   - int x = 1;
     int y = 2;

   Penjelasan:

   Program mendeklarasikan dua variabel bertipe int yaitu x bernilai 1 dan y bernilai 2. Kedua variabel ini akan digunakan sebagai operand dalam ekspresi ternary untuk dibandingkan nilainya.

   - System.out.print("x = "+ x + "\n");
     System.out.print("y = "+ y + "\n");

   Penjelasan:

   Dua baris ini mencetak nilai awal kedua variabel ke console menggunakan System.out.print dengan karakter \n sebagai pemindah baris manual. Ini memperlihatkan nilai yang akan diproses sebelum ekspresi ternary dijalankan.

- System.out.print("hasil ekspresi = (x<y)?x:y = " + ((x < y) ? x : y));

Penjelasan:
Cara kerja ekspresi (x < y) ? x : y :
Pertama Java mengevaluasi kondisi x < y yaitu 1 < 2. Karena kondisi ini bernilai true, maka ekspresi mengembalikan nilai di sebelah kiri tanda titik dua yaitu x yang bernilai 1. Jika kondisi bernilai false, maka yang dikembalikan adalah nilai di sebelah kanan titik dua yaitu y. Dengan kata lain, ekspresi ini berfungsi mencari nilai minimum dari dua bilangan.
Penjelasan tanda kurung ganda (( )) pada kode ((x < y) ? x : y) seperti yang dijelaskan dalam komentar program, tanda kurung paling luar digunakan untuk menyatakan satu kesatuan pernyataan agar Java tidak salah menginterpretasikan operator + sebagai string concatenation dengan bagian-bagian ternary secara terpisah. Tanda kurung dalam (x < y) adalah kondisi ternary itu sendiri.

Output:

```
x = 1
y = 2
hasil ekspresi = (x<y)?x:y = 1
```

9. Penjelasan program nomor 9
Program ini bertujuan untuk mendemonstrasikan perilaku operasi pembagian dalam Java berdasarkan tipe data yang digunakan. Program memperlihatkan perbedaan hasil pembagian ketika menggunakan tipe int vs float, serta tiga cara berbeda untuk menghasilkan nilai desimal dari pembagian bilangan bulat yaitu melalui penugasan ke float, explicit casting, dan perubahan nilai variabel. Konsep ini sangat penting karena pembagian integer di Java memotong bagian desimalnya secara otomatis.
   - public class Ekspresi1 {
     public static void main(String[] args) {

Penjelasan:

- public class Ekspresi1 { : Wadah utama program. Nama kelas harus sama dengan nama file. Keyword public berarti kelas ini bisa diakses dari mana saja.
- public static void main(String[] args) { : Titik awal (entry point) saat program dijalankan. Tanpa ini, kode tidak bisa dieksekusi

- int x = 1; int y = 2; float fx; float fy;

Penjelasan:
Empat variabel dideklarasikan dalam satu baris. x dan y bertipe int dengan nilai awal 1 dan 2. fx dan fy bertipe float namun belum diinisialisasi, keduanya akan digunakan sebagai

versi float dari x dan y. Penulisan beberapa deklarasi dalam satu baris seperti ini valid di Java namun kurang disarankan untuk keterbacaan.

- System.out.print("x/y (format integer) = "+ x/y);
  System.out.print("\nx/y (format float) = "+ x/y);

Penjelasan:
Kedua baris ini mencetak hasil x/y yaitu 1/2. Karena kedua operand bertipe int, Java melakukan integer division yang secara otomatis memotong bagian desimal hasilnya. Hasil 1/2 yang seharusnya 0.5 menjadi 0 karena bagian desimal .5 dibuang. Meskipun komentar menyebutkan "format float", operasinya tetap integer division karena tipe variabelnya tidak berubah. Kedua baris menghasilkan output yang sama persis yaitu 0.

- fx = x;
  fy = y;
  System.out.print("\nx/y (format integer) = "+ fx/fy);
  System.out.print("\nx/y (format float) = "+ fx/fy);

Penjelasan:
Di sini nilai x dan y dipindahkan ke variabel fx dan fy yang bertipe float. Proses ini adalah widening conversion otomatis (implicit casting) dari int ke float sehingga fx menjadi 1.0f dan fy menjadi 2.0f. Ketika operasi fx/fy dilakukan, Java mengenali kedua operand sebagai float sehingga melakukan floating point division yang menghasilkan nilai desimal. Hasilnya 1.0/2.0 = 0.5. Sama seperti bagian 1, kedua baris mencetak hasil yang sama karena operasinya identik.

- System.out.print("\nfloat(x)/float(y) (format integer)="+ (float)x/(float)y);
  System.out.print("\nfloat(x)/float(y) (format float) ="+ (float)x/(float)y);

Penjelasan:
Cara ketiga menggunakan explicit casting langsung pada operand tanpa menggunakan variabel perantara. Operator (float) diterapkan langsung ke x dan y saat operasi berlangsung. (float)x mengonversi int 1 menjadi float 1.0f secara sementara tanpa mengubah nilai variabel x itu sendiri, begitu pula (float)y mengonversi 2 menjadi 2.0f. Hasilnya pembagian dilakukan dalam mode float menghasilkan 0.5. Ini adalah cara paling ringkas dan umum digunakan untuk menghindari integer division tanpa perlu variabel tambahan. Sekali lagi, kedua baris menghasilkan output sama.

- x = 10; y = 3;
  System.out.print("\nx/y (format integer) = "+ x/y);
  System.out.print("\nx/y (format float) = "+ x/y);

Penjelasan:
Nilai x dan y diubah menjadi 10 dan 3. Pembagian 10/3 kembali dilakukan dengan tipe int sehingga hasilnya adalah integer division. Hasil matematis 10/3 = 3.333... dipotong menjadi 3. Sisa pembagian 1 dibuang begitu saja. Jika ingin mendapatkan sisa pembagian, Java menyediakan operator modulo % yaitu 10 % 3 = 1.

Output:

```
x/y (format integer) = 0
x/y (format float) = 0
x/y (format integer) = 0.5
x/y (format float) = 0.5
float(x)/float(y) (format integer)=0.5
float(x)/float(y) (format float) =0.5
x/y (format integer) = 3
x/y (format float = 3
```

10. Penjelasan program nomor 10

Program ini merupakan program paling dasar dalam Java yang bertujuan untuk mendemonstrasikan cara mencetak teks ke layar (console output) menggunakan dua method berbeda yaitu System.out.print() dan System.out.println(). Program juga memperkenalkan penggunaan escape sequence \n sebagai karakter pindah baris manual.

- public class PrintHello {
  public static void main(String[] args) {

Penjelasan:

- public class PrintHello{ : Wadah utama program. Nama kelas harus sama dengan nama file. Keyword public berarti kelas ini bisa diakses dari mana saja.
- public static void main(String[] args) { : Titik awal (entry point) saat program dijalankan. Tanpa ini, kode tidak bisa dieksekusi

- System.out.print("Hello");

Penjelasan:

Baris ini mencetak teks Hello ke console menggunakan method print(). Karakteristik utama print() adalah tidak menambahkan baris baru secara otomatis di akhir teks. Artinya kursor tetap berada di posisi setelah huruf o dari kata Hello, sehingga teks berikutnya akan langsung disambung di baris yang sama.

- System.out.print("\nHello ");

Penjelasan:

Baris ini mencetak \nHello namun karakter \n adalah escape sequence yang tidak dicetak sebagai teks literal melainkan diinterpretasikan sebagai perintah pindah baris (newline). Ketika dieksekusi, \n memindahkan kursor ke baris baru terlebih dahulu, baru kemudian mencetak Hello (dengan spasi di belakang). Karena method yang digunakan tetap print(), kursor kembali berhenti setelah spasi tanpa pindah baris otomatis.

- System.out.println("World");
  System.out.println("Welcome");

Penjelasan:

Kedua baris ini menggunakan println() yang merupakan singkatan dari print line. Perbedaan utamanya dari print() adalah println() secara otomatis menambahkan karakter newline di akhir setiap teks yang dicetak, sehingga teks berikutnya selalu dimulai dari baris baru. println("World") disambung langsung setelah Hello (dari baris sebelumnya) karena print() tidak memindahkan kursor. Lalu println("Welcome") dicetak di baris baru karena println() sebelumnya sudah menambahkan newline otomatis.

Output:

```
Hello
Hello World
Welcome
```

11. Penjelasan program nomor 11

- public class Incr {
  public static void main(String[] args) {

Penjelasan:

- public class Incr{ : Wadah utama program. Nama kelas harus sama dengan nama file. Keyword public berarti kelas ini bisa diakses dari mana saja.
- public static void main(String[] args) { : Titik awal (entry point) saat program dijalankan. Tanpa ini, kode tidak bisa dieksekusi

- int i, j;

Penjelasan:

Dua variabel bertipe int dideklarasikan sekaligus dalam satu baris tanpa inisialisasi. Variabel i akan menjadi subjek utama operasi increment, sedangkan j digunakan untuk menangkap dan menyimpan hasil dari operasi post-increment pada i.

- i = 3;
  j = i++;

Penjelasan:

Variabel i diinisialisasi dengan nilai 3. Kemudian baris j = i++ adalah inti pertama dari program ini. Operator post-increment i++ memiliki aturan bahwa nilai i yang lama digunakan terlebih dahulu untuk operasi yang sedang berjalan, baru kemudian nilai i ditambah 1. Urutan eksekusinya adalah sebagai berikut:

i = 3
j = i++ → j mengambil nilai i yang lama (3)
lalu i ditambah 1 menjadi 4
Hasil : j = 3, i = 4
Jadi setelah baris ini selesai, j bernilai 3 dan i sudah berubah menjadi 4.

- System.out.println("Nilai i : " + (++i) + "\nNilai j : " + j);

Penjelasan:
Di dalam println terdapat ekspresi pre-increment ++i. Berbeda dari post-increment, operator ++i menambahkan nilai i terlebih dahulu sebelum nilai tersebut digunakan dalam ekspresi. Pada saat baris ini dieksekusi, nilai i sudah 4 (hasil post-increment sebelumnya). Urutan eksekusinya adalah:
i saat ini = 4
++i → i ditambah 1 dulu menjadi 5
lalu nilai 5 digunakan dalam println
Hasil : i = 5, dicetak 5
Tanda kurung (++i) ditulis sesuai komentar pada program Ekspresi.java sebelumnya, yaitu untuk menyatakan satu kesatuan ekspresi agar operator ++ tidak salah diinterpretasikan bersama operator + konkatenasi String di sekitarnya.

Output:

```
Nilai i : 5
Nilai j : 3
```

12. Penjelasan program nomor 12
Program ini bertujuan untuk mendemonstrasikan Operator Bitwise dalam Java, yaitu operator yang bekerja langsung pada level bit biner dari nilai integer. Berbeda dari operator aritmatika biasa yang bekerja pada nilai desimalnya, operator bitwise memanipulasi setiap bit secara individual. Program memperlihatkan empat jenis operasi bitwise yaitu AND (&), NOT (~), Left Shift (<<), dan Right Shift (>>) beserta hasil konversinya ke nilai desimal.

- public class Oper1 {
  public static void main(String[] args) {

Penjelasan:

- public class Oper1 { : Wadah utama program. Nama kelas harus sama dengan nama file. Keyword public berarti kelas ini bisa diakses dari mana saja.
- public static void main(String[] args) { : Titik awal (entry point) saat program dijalankan. Tanpa ini, kode tidak bisa dieksekusi

- int n = 10; /* 1010 */

```
int x = 1;  /* 1    */
int y = 2;  /* 10   */
```

Penjelasan:

Tiga variabel bertipe int dideklarasikan beserta representasi binernya dalam komentar. Memahami representasi biner ini adalah kunci untuk memahami seluruh program. Dalam sistem biner, setiap angka direpresentasikan sebagai kombinasi bit 0 dan 1.

Desimal 10 = Biner 0000 1010

Desimal 1 = Biner 0000 0001

Desimal 2 = Biner 0000 0010

Perlu diingat bahwa tipe int di Java menggunakan 32 bit, namun untuk kemudahan penjelasan hanya bit-bit yang relevan yang ditampilkan.

- System.out.println("n & 8 = "+ (n & 8)); /* 1010 AND 1000 */

Penjelasan:

Operator & membandingkan setiap pasangan bit dari dua operand. Hasilnya 1 hanya jika kedua bit bernilai 1, selain itu hasilnya 0. Ini mengikuti tabel kebenaran AND:

- System.out.println("x & ~8 = "+ (x & ~8)); /* 1 AND 0111 */

Penjelasan:

Baris ini menggunakan dua operator sekaligus. Operator ~ adalah bitwise NOT atau komplemen yang membalik semua bit, setiap 0 menjadi 1 dan setiap 1 menjadi 0. Karena int Java adalah 32 bit, ~8 membalik semua 32 bit dari angka 8.

- System.out.println("y << 2 = "+ (y << 2)); /* 10 ==> 1000 = 8 */

Penjelasan:

Operator << menggeser semua bit ke kiri sebanyak jumlah yang ditentukan. Bit yang keluar dari sisi kiri dibuang, dan posisi yang kosong di sisi kanan diisi dengan 0. Setiap geser kiri 1 bit sama dengan mengalikan dengan 2. Sehingga y << 2 sama dengan $y \times 2^2 = 2 \times 4 = 8$. Left shift sering digunakan sebagai perkalian cepat dengan pangkat 2 karena lebih efisien dari operasi perkalian biasa.

- System.out.println("y >> 3 = "+ (y >> 3)); /* 10 ==> 0000 = 0 */

Penjelasan:

Operator >> menggeser semua bit ke kanan sebanyak jumlah yang ditentukan. Bit yang keluar dari sisi kanan dibuang, dan posisi yang kosong di sisi kiri diisi dengan bit tanda (sign bit) yaitu 0 untuk bilangan positif dan 1 untuk bilangan negatif. Inilah yang membedakan >> (arithmetic right shift) dari >>> (unsigned right shift). Setiap geser kanan 1 bit sama dengan membagi dengan 2 (pembagian integer, bagian desimal dibuang). Karena 2 hanya memiliki 1 bit aktif dan digeser 3 posisi, semua bit aktif habis terbuang sehingga hasilnya 0.

Output:

```
n = 10
x = 1
y = 2
n & 8 = 8
x & ~ 8 = 1
y << 2 = 8
y >> 3 = 0
```

13. Penjelasan program nomor 13

Program ini melanjutkan konsep operator bitwise dari codingan sebelumnya namun menggunakan tipe data char sebagai operand, serta memperkenalkan operator XOR (^), OR (|), NOT (~), dan penggunaan Math.pow() untuk pemangkatan sebagai pembanding dengan operator ^ yang sering disalahartikan sebagai pangkat.

- public class Oper2 {
  public static void main(String[] args) {

Penjelasan:

- public class Oper2{ : Wadah utama program. Nama kelas harus sama dengan nama file. Keyword public berarti kelas ini bisa diakses dari mana saja.
- public static void main(String[] args) { : Titik awal (entry point) saat program dijalankan. Tanpa ini, kode tidak bisa dieksekusi

- char i, j;
  i = 3; /* 00000011 dalam biner */
  j = 4; /* 00000100 dalam biner */

Penjelasan:

Variabel i dan j bertipe char namun diisi dengan nilai integer 3 dan 4. Seperti yang dipelajari pada program sebelumnya, char dapat menerima nilai integer dan menyimpannya sebagai nilai Unicode. Nilai 3 dan 4 bukan karakter yang terlihat (karakter kontrol) namun tetap valid sebagai nilai char. Dalam biner 8-bit:

i = 3 → 0000 0011

j = 4 → 0000 0100

- System.out.println("i = "+ (int) i);
  System.out.println("j = "+ j);

Penjelasan:

Baris pertama menggunakan explicit cast (int) untuk mencetak nilai numerik dari i. Tanpa cast ini, Java akan mencoba mencetak i sebagai karakter Unicode yang tidak terlihat (karakter kontrol). Baris kedua mencetak j langsung tanpa cast, namun karena digabungkan dengan String menggunakan +, Java otomatis mengonversinya ke representasi numerik sehingga tetap mencetak 4.

- System.out.println("i & j = "+ (i & j));
  /* 0: 00000000 dalam biner */

Penjelasan:

i = 3 → 0000 0011
j = 4 → 0000 0100

---

i & j → 0000 0000 = 0

Tidak ada posisi bit yang keduanya bernilai 1 secara bersamaan sehingga semua bit hasil adalah 0. Output: i & j = 0

- System.out.println("i | j = "+ (i | j));
  /* 7: 00000111 biner */

Penjelasan:

Operator | menghasilkan 1 jika salah satu atau kedua bit bernilai 1:

i = 3 → 0000 0011
j = 4 → 0000 0100

---

i | j → 0000 0111 = 7

Bit dari i dan j digabungkan sehingga semua posisi yang memiliki 1 di salah satu operand ikut menjadi 1. Output: i | j = 7

- System.out.println("i ^ j = "+ (i ^ j));
  /* 7: 00000111 biner */

Penjelasan:

Operator ^ (XOR / Exclusive OR) menghasilkan 1 hanya jika kedua bit berbeda:

Aturan XOR : 0 ^ 0 = 0 (sama → 0)
0 ^ 1 = 1 (beda → 1)
1 ^ 0 = 1 (beda → 1)
1 ^ 1 = 0 (sama → 0)

i = 3 → 0000 0011
j = 4 → 0000 0100

---

i ^ j → 0000 0111 = 7

Karena tidak ada posisi bit yang keduanya 1 secara bersamaan (sama seperti AND hasilnya 0), semua bit yang berbeda menghasilkan 1. Output: i ^ j = 7

- System.out.println(Math.pow(i, j));

Penjelasan:

Math adalah class bawaan Java dari package java.lang yang berisi kumpulan method matematika statis. Method pow(a, b) menghitung nilai a dipangkatkan b (a^b) dan mengembalikan hasil bertipe double. Di sini i=3 dan j=4 sehingga Math.pow(3, 4) = 3^4 = 81.0.

Karena i dan j bertipe char, Java secara otomatis mempromosikan nilainya ke tipe yang sesuai sebelum diproses Math.pow().

- System.out.println("~i = "+ ~i);
  /* -4: 11111100 biner */

Penjelasan:

Operator ~ membalik seluruh 32 bit dari nilai integer. Java menggunakan sistem two's complement untuk merepresentasikan bilangan negatif:

i = 3 → 0000 0000 0000 0000 0000 0000 0000 0011

~i → 1111 1111 1111 1111 1111 1111 1111 1100

= -4 (dalam sistem two's complement)

Rumus umum NOT bitwise: ~n = -(n+1), sehingga ~3 = -4. Output: ~i = -4

Output:

```
i = 3
j =
i & j = 0
i | j = 7
i ^ j = 7
81.0
~i = -4
```

14. Penjelasan program nomor 14

    Program ini bertujuan mendemonstrasikan perbedaan antara Logical Operator (&&, ||) dan Bitwise Operator (&, |) ketika diterapkan pada nilai boolean. Keduanya menghasilkan nilai yang sama untuk boolean, namun memiliki perbedaan mendasar dalam hal short-circuit evaluation.

    - public class Oper3 {
      public static void main(String[] args) {

    Penjelasan:

    - public class Oper3 { : Wadah utama program. Nama kelas harus sama dengan nama file. Keyword public berarti kelas ini bisa diakses dari mana saja.
    - public static void main(String[] args) { : Titik awal (entry point) saat program dijalankan. Tanpa ini, kode tidak bisa dieksekusi

    - if (true && true) { System.out.println(true && true); }

    Penjelasan:

Operator && adalah logical AND dengan short-circuit evaluation. Artinya jika operand pertama sudah false, operand kedua tidak dievaluasi sama sekali karena hasilnya sudah pasti false. Di sini kedua operand true sehingga hasilnya true. Output: true

- if (true & true) { System.out.println(true & true); }

Penjelasan:

Operator & pada boolean adalah bitwise AND non-short-circuit. Berbeda dari &&, operator ini selalu mengevaluasi kedua operand meskipun operand pertama sudah false. Untuk nilai boolean hasilnya sama dengan &&, namun lebih lambat karena tidak ada optimasi short-circuit. Output: true
Java

- if (true) { System.out.println(true); }

Penjelasan:

Kondisi if (true) selalu terpenuhi, mencetak nilai boolean true secara langsung. Output: **true**

- if (true || true) { System.out.println(true || true); }

Penjelasan:
Operator || adalah logical OR dengan short-circuit evaluation. Jika operand pertama sudah true, operand kedua tidak dievaluasi karena hasilnya sudah pasti true. Output: true

- if (true | false) { System.out.println(true | false); }

Penjelasan:
Operator | pada boolean adalah bitwise OR non-short-circuit yang selalu mengevaluasi kedua operand. true | false menghasilkan true karena minimal satu operand bernilai true. Output: true

Output:

```
true
true
true
true
true
```

15. Penjelasan program nomor 15
Program ini menggabungkan tiga konsep sekaligus yaitu operator ternary, post-increment, dan perbandingan nilai char untuk mendemonstrasikan bagaimana ketiga konsep tersebut saling berinteraksi dalam satu ekspresi. Program juga menekankan pentingnya memahami urutan evaluasi ekspresi ketika increment dan ternary digabungkan.

- public class Oper4{

public static void main(String[] args) {

Penjelasan:

- public class Oper4{ : Wadah utama program. Nama kelas harus sama dengan nama file. Keyword public berarti kelas ini bisa diakses dari mana saja.
- public static void main(String[] args) { : Titik awal (entry point) saat program dijalankan. Tanpa ini, kode tidak bisa dieksekusi

- int i = 0;
  int j = 0;
  char c = 8; char d = 10;

Penjelasan:
Variabel i dan j bertipe int masing-masing bernilai 0. Komentar dalam kode menegaskan bahwa int i, j = 0 tidak menginisialisasi keduanya ke 0 — hanya j yang terinisialisasi, sedangkan i tidak. Itulah mengapa keduanya dideklarasikan terpisah. Variabel c dan d bertipe char berisi nilai integer 8 dan 10 yang merepresentasikan karakter kontrol Unicode.

- int e = (((int)c > (int)d) ? c : d);

Penjelasan:
Ekspresi ini menggunakan ternary untuk mencari nilai maksimum antara c dan d. Cast (int) dilakukan sebelum perbandingan untuk memastikan Java membandingkan nilai numeriknya bukan representasi karakternya. Evaluasinya:
(int)c = 8
(int)d = 10
Kondisi : 8 > 10 → false
Hasil : d dipilih → e = 10

- int k = ((i > j) ? i : j);

Penjelasan:
Dengan i=0 dan j=0, kondisi i > j yaitu 0 > 0 adalah false, sehingga j dipilih. Karena i dan j sama-sama 0, nilai k = 0.

Output:

```
Nilai e = 10
Nilai k = 0
Nilai k = 4
```

16. Penjelasan program nomor 16
Program ini merupakan demonstrasi komprehensif seluruh jenis operator dasar dalam Java menggunakan tiga tipe data primitif sekaligus yaitu boolean, int, dan float. Program

mencakup empat kelompok operasi yaitu logika boolean, aritmatika integer, aritmatika float, dan relasional pada kedua tipe numerik sehingga menjadi referensi lengkap penggunaan operator di Java.

- public class Oprator{
  public static void main(String[] args) {

Penjelasan:

- public class Oprator{ : Wadah utama program. Nama kelas harus sama dengan nama file. Keyword public berarti kelas ini bisa diakses dari mana saja.
- public static void main(String[] args) { : Titik awal (entry point) saat program dijalankan. Tanpa ini, kode tidak bisa dieksekusi

- boolean Bool1, Bool2, TF;
  int i, j, hsl;
  float x, y, res;

Penjelasan:
Tiga kelompok variabel dideklarasikan sesuai tipe datanya. Variabel boolean hanya dapat menyimpan dua nilai yaitu true atau false, digunakan untuk menyimpan hasil operasi logika dan relasional. Variabel int menyimpan bilangan bulat untuk operasi aritmatika integer. Variabel float menyimpan bilangan desimal 32-bit untuk operasi aritmatika pecahan. Semua variabel belum diinisialisasi pada tahap deklarasi.

- Bool1 = true; Bool2 = false;
  TF = Bool1 && Bool2; // AND
  TF = Bool1 || Bool2; // OR
  TF = !Bool1; // NOT
  TF = Bool1 ^ Bool2; // XOR

Penejelasan:
Keempat operator logika bekerja pada nilai boolean dan menghasilkan boolean. Operator && (AND) menghasilkan true hanya jika kedua operand true. Operator || (OR) menghasilkan true jika minimal satu operand true. Operator ! (NOT) membalik nilai boolean, true menjadi false dan sebaliknya. Operator ^ (XOR) menghasilkan true hanya jika kedua operand berbeda nilainya.

- i = 5; j = 2;
  hsl = i + j; // 7
  hsl = i - j; // 3
  hsl = i * j; // 10
  hsl = i / j; // 2 (bukan 2.5!)
  hsl = i % j; // 1

Penjelasan:

Semua operasi menggunakan variabel int sehingga hasilnya selalu bilangan bulat. Yang paling perlu diperhatikan adalah pembagian i/j = 5/2 yang menghasilkan 2 bukan 2.5 karena Java memotong bagian desimal pada integer division. Operator modulo % menghasilkan sisa pembagian, 5%2 = 1 karena 5 = (2×2) + 1.

- x = 5.5f; y = 2.2f;
  res = x + y; // 7.7
  res = x - y; // 3.3
  res = x * y; // 12.1
  res = x / y; // 2.5

Penjelasan:
Nilai diubah menjadi 5.5f dan 2.2f agar hasil operasi lebih jelas menunjukkan perbedaan dengan integer. Karena kedua operand bertipe float, pembagian x/y menghasilkan nilai desimal 2.5 tidak seperti integer division. Suffix f wajib ditulis untuk membedakan literal float dari double.

- // Integer
  TF = (i == j); // false : 5 tidak sama dengan 2
  TF = (i != j); // true : 5 memang tidak sama dengan 2
  TF = (i < j); // false : 5 tidak lebih kecil dari 2
  TF = (i > j); // true : 5 lebih besar dari 2
  TF = (i <= j); // false : 5 tidak lebih kecil atau sama dengan 2
  TF = (i >= j); // true : 5 lebih besar dari 2

  // Float (logika sama, hanya tipe data berbeda)
  TF = (x == y); // false
  TF = (x != y); // true
  TF = (x < y); // false
  TF = (x > y); // true
  TF = (x <= y); // false
  TF = (x >= y); // true

Penjelasan:
Semua operator relasional membandingkan dua nilai dan selalu menghasilkan boolean (true/false). Operator relasional untuk float bekerja dengan cara yang sama seperti untuk int, namun perlu diwaspadai penggunaan == pada float/double karena representasi floating point tidak selalu tepat sehingga perbandingan == bisa menghasilkan false meskipun secara matematis nilainya sama.